

Nix Shells for Node Projects

Rohit Goswami · 2020-08-23 · tools · nix · workflow · node

Background

As a prelude to writing up the details of how this site is generated, I realized I should write up a `nix` oriented workflow for `node` packages.

Tooling and Idea

The basic concepts are:

- Use `npm` to generate a `package-lock.json` file
- Use `node2nix` in a shell to generate a set of `nix` derivations
- Enter a shell environment with the `nix` inputs
- Profit

However, the nuances of this are a bit annoying at first.

Packaging Requirements

We will use the standard `npm` installation method at first, but since we shouldn't keep installing and removing things, so we need a way to modify `package.json` without running `npm` and will therefore add `add-dependency`.

```
npm install add-dependency
```

Setting up Node2Nix

We will first clean the directory of what we do not need.

```
rm -rf default.nix node-env.nix node-packages.nix node_modules
```

Now we can enter a shell with `node2nix` and generate files for the node packages.

```
nix-shell -p 'nodePackages.node2nix'  
node2nix -l package-lock.json
```

A Nix Environment

We will use the standard setup described in [the tutorial post](#):

```
nix-env -i niv lorri  
niv init  
niv update nixpkgs -b nixpkgs-unstable
```

This is to be in conjunction with the following `shell.nix` file [^fn:1].

```
{ sources ? import ./nix/sources.nix };
let
  pkgs = import sources.nixpkgs { };
  nodeEnv = pkgs.callPackage ./node-env.nix { };
  nodePackages = pkgs.callPackage ./node-packages.nix {
    globalBuildInputs = with pkgs; [ zsh ];
    inherit nodeEnv;
  };
in nodePackages.shell
```

Note that we have overridden the `nodePackages` shell which is defined in the files created by `node2nix`.

We can now enter the environment and setup `node_modules` [^fn:2].

```
nix-shell
ln -s $NODE_PATH node_modules
```

Updates

Unfortunately, this setup is a little fragile to updates. We will need to exit and re-create the setup. Note that we are removing the lock file now as well.

```
# In the nix-shell
add-dependencies babel-loader @babel/core @babel/preset-env core-js @babel/plugin-transform-regenerator
# Do not run in nix-shell
rm -rf default.nix node-env.nix node-packages.nix node_modules package-lock.json
# Update in a line
nix-shell -p 'nodePackages.node2nix' --run 'node2nix package.json'
```

The single line update mechanism can be run in the `nix-shell` itself. This makes things marginally less painful.

Conclusions

This has been a short introduction to working with the `nix-shell` ecosystem. It isn't as fast as working with the normal setup, and it is a pretty annoying workflow. Given that most CI setups have good support for caching `npm` dependencies, it doesn't seem worthwhile at the moment.